Bảng Tổng Hợp Vi Phạm Design Principles — Auction Bidding Flow

> **Phạm vi phân tích:** `product.route.js`, `review.model.js`, `auctionEndNotifier.js` và kiến trúc tổng thể của module đấu giá.

Bảng Vi Phạm

STT	Location	Nguyên tắc vi phạm	Mô tả vi phạm	Tác động đến Maintainability / Exten
1	`product.route.js` **Toàn bộ file**	**SRP**	File hiện tại xử lý đồng thời 8 domain khác nhau: HTTP routing, bid validation, auto-bid algorithm, auction state machine, DB access, email notification, order creation, rating/reputation. Mỗi domain là một "lý do độc lập để thay đổi" — vi phạm nguyên tắc một module chỉ có một trách nhiệm.	Bất kỳ thay đổi nhỏ nào (đổi email tem thêm payment gateway, cập nhật rule t rating) đều buộc phải mở file ~1000+ c này. Rủi ro regression cao vì các domai không có boundary, thay đổi ở một cor có thể vô tình ảnh hưởng concern khác coverage gần như không thể đạt do kh unit nhỏ để isolate.

2	`product.route.js`
	`GET /detail`
	`GET /complete-order`
		DRY	Khối logic 5 nhánh xác định `productStatus` (`SOLD` / `CANCELLED` / `PENDING` / `EXPIRED` / `ACTIVE`) được viết lại nguyên vẹn ở hai route handler khác nhau. Không có hàm chung nào được trích xuất.	Khi thêm state mới (ví dụ `DISPUTED`), l trình viên phải nhớ sửa **cả hai chỗ**. Nếu sửa một chỗ, hai route trả về kết quả kh nhau cho cùng một product — data inconsistency nhìn thấy được từ phía us là dạng bug ki古典 do copy-paste.

3	`product.route.js` `GET /detail`	**SRP**	Handler `GET /detail` là một read operation nhưng chứa một **write side effect**: khi phát hiện auction đã hết hạn (`endDate <= now`) mà `closed_at` chưa được set, nó tự động gọi `productModel.updateProduct(productId, { closed_at: endDate })`. Đây là một state transition (`ACTIVE → PENDING/EXPIRED`) xảy ra bên trong một GET request.	GET requests theo convention HTTP là idempotent và không có side effect. Sid effect ẩn này: (1) không thể cache respc (2) làm khó debug vì state thay đổi khô trigger, (3) nếu load balancer replicate request, transition có thể bị trigger nhiề (4) unit test cho "read detail" sẽ vô tình luôn "write state" — làm test dirty.
4	`product.route.js` `POST /bid`	**KISS**	Handler `POST /bid` thực hiện tuần tự 15 bước trong một hàm duy nhất (~400 dòng): acquire DB lock, validate 5 điều kiện eligibility, validate amount/increment, apply auto-extend, chạy auto-bid algorithm (3 case lồng nhau), kiểm tra buy-now trigger, update DB (3 table), gửi 3 email song song, build response message (4 nhánh), redirect với flash message. Không có bước nào được đặt tên riêng hay tách hàm.	Không thể đọc hiểu trong <30 giây. Kh thể test một bước độc lập (ví dụ: test au algorithm mà không cần setup toàn bộ pipeline). Debugging yêu cầu đọc hàng dòng để tìm điểm lỗi. Thêm một validat mới (ví dụ: rate limit) đòi hỏi hiểu toàn context.

5	`product.route.js`
	`POST /bid`
	(auto-bidding algorithm)
		OCP	Thuật toán auto-bid được implement dưới dạng `if (case === 0) { ... } else if (case === 1) { ... } else { ... }` inline trong handler. Mỗi case là một "chiến lược" bidding khác nhau. Khi cần thêm loại bid mới (ví dụ: sealed bid, reserve price bid), lập trình viên phải **sửa trực tiếp** khối if/else đang chạy trong production — vi phạm "closed for modification".	Sửa một case có nguy cơ break case kh cùng nằm trong một khối logic. Không A/B test một algorithm mới mà không toàn bộ route file. Mỗi lần thêm bid typ review lại toàn bộ handler để đảm bảo có interaction không mong muốn.

6	`product.route.js` (auction status: `SOLD/CANCELLED/PENDING/EXPIRED/ACTIVE`)	**OCP**	Logic 5 nhánh if/else xác định auction state được viết trực tiếp trong route handler, không phải trong một state machine module. Mỗi lần thêm state mới (ví dụ: `PAUSED`, `DISPUTED`) lập trình viên phải: tìm tất cả các chỗ có khối if/else này, sửa từng chỗ, đảm bảo tất cả nhất quán — tức là **phải modify code cũ** để extend.	State logic scattered làm cho việc reaso system behavior tốn công. Không có ch nhất để nhìn tổng quan "auction có thể những state nào và khi nào". Thêm stat có thể bị bỏ sót ở một trong nhiều chỗ, đến inconsistency.
7	`product.route.js` `POST /bid` (email HTML)	**DRY**	Ba email được gửi trong `POST /bid` (cho seller, current bidder, previous bidder) đều tự viết lại toàn bộ HTML boilerplate: cùng wrapper `<div>`, cùng `font-family`, cùng gradient header, cùng footer disclaimer. Chỉ khác nhau ở title và body content — nhưng không có template builder chung.	Đổi màu brand, font, hoặc cấu trúc layo email phải sửa ở 3 chỗ. Nếu thêm loại e thứ 4, lập trình viên sẽ copy-paste thêm lần nữa. Khó đảm bảo visual consistenc các email khi code bị phân tán.

STT	Location	Nguyên tắc vi phạm	Mô tả vi phạm	Tác động đến Maintainability / Exten
8	`product.route.js` Dependencies: `product.model.js`, `mailer.js`, `db.js`, `order.model.js`, `review.model.js`	**DIP**	`product.route.js` (high-level module điều phối business flow) import trực tiếp các concrete implementations: DB models (Knex/PostgreSQL), mailer transport (Nodemailer/SMTP), DB connection pool. Không có abstraction layer ở giữa — route biết rõ đang dùng ORM nào, mail service nào.	Đổi ORM từ Knex sang Prisma: phải sửa file. Đổi mail provider từ SMTP sang Se phải sửa route file. Unit test route hand buộc phải mock toàn bộ DB layer thay mock service interface — brittle tests. Coupling cao giữa HTTP layer và infrast layer.

9	`product.route.js`
	`GET /complete-order`
	`POST /buy-now`
		SRP (Bidding ↔ Order coupling)	Hai domain biệt lập — Auction và Order — bị trộn lẫn: (1) `GET /complete-order` chứa fallback logic auto-create Order nếu chưa tồn tại (`if (!order) await orderModel.createOrder(...)`), thay vì Order được tạo ngay tại thời điểm auction kết thúc. (2) `POST /buy-now` kết thúc auction nhưng **không tạo Order** — Order chỉ được tạo khi user navigate đến `/complete-order`. Hai route trong Auction domain đang làm công việc của Order domain theo cách inconsistent.	Nếu user bỏ qua bước `/complete-or` (bị interrupt, đóng tab), Order không b được tạo — dữ liệu business bị mất. Bu và regular-bid có hành vi khác nhau về điểm tạo Order — inconsistency có thể đến bug phân giải phức tạp. Auction se phải biết về Order schema — coupling domain không đáng có.

10	`product.route.js` `GET /bid-history/:productId`	**YAGNI**	Sau khối `try/catch` đã `res.json()` và kết thúc response, file còn chứa ~10 dòng code render-based hoàn toàn unreachable: load `productModel.findByProductId()`, `findRelatedProducts()`, rồi `res.render('vwProduct/details', ...)`. Đây là code từ phiên bản cũ (render-based) chưa được xóa khi route chuyển sang JSON API. Ngoài unreachable, nếu code này được trigger bằng cách nào đó, sẽ gây lỗi **"Cannot set headers after they are sent"** và `productId` ra khỏi scope.	Dead code gây confused cho developer file (không rõ ý định là render hay JSON `productModel.findRelatedProdu` và `productModel.findByProductI` được import/gọi không cần thiết, tiêu t nguyên nếu code path không đúng. Tăr cognitive load khi đọc file.
11	`review.model.js` `create()` vs `createReview()`	**DRY + YAGNI**	Model export hai hàm thực hiện cùng một INSERT operation: `createReview(reviewData)` nhận field `reviewData.reviewee_id`; `create(data)` nhận field `data.reviewed_user_id` nhưng map thành cùng column `reviewee_id` trong DB. Caller (`product.route.js`) chỉ dùng `create()` — `createReview()` không có call site nào trong toàn bộ codebase.	Caller phải biết "nên dùng hàm nào" và field là gì" — knowledge không được e trong module. Nếu ai đó dùng `createReview()` với field `reviewed_user_id` (nhầm sang conv của `create()`), data sẽ bị null hoặc er hàm có thể drift về behavior theo thời g cùng làm một việc.
12	`review.model.js` `updateReview()` vs `updateByReviewerAndProduct()`	**DRY + YAGNI**	Tương tự vi phạm trên nhưng cho UPDATE path: `updateReview(reviewer_id, reviewee_id, product_id, updateData)` WHERE 3 columns; `updateByReviewerAndProduct(reviewer_id, product_id, updateData)` WHERE 2 columns. Vì `(reviewer_id, product_id)` đã đủ unique (1 người chỉ review 1 lần / 1 sản phẩm), `reviewee_id` trong WHERE của `updateReview` là thừa. Call site chỉ dùng `updateByReviewerAndProduct()`, `updateReview()` không có call site.	Tương tự: dual API tạo ambiguity cho c `updateReview` với reviewee_id sai sẽ fail (UPDATE 0 rows) thay vì lỗi rõ ràng - là loại bug rất khó debug. Dead code là phình module không cần thiết.
13	`product.route.js` `GET /detail` (multi-concern)	**KISS**	Handler `GET /detail` load song song 6 data sources, tính pagination comments, tính seller rating, tính bidder rating, xác định productStatus (5 nhánh), kiểm tra access control, detect và trigger auto-close side effect, rồi build một viewmodel với 15+ fields — tất cả trong một hàm không có tên phụ nào.	Không thể test riêng từng phần (ví dụ: t "build viewmodel" mà không mock tất data sources). Khi một trong 6 queries không có metric để biết query nào gây bottleneck. Thêm một điều kiện access mới yêu cầu đọc hiểu toàn bộ handler context.
14	`auctionEndNotifier.js`	**DRY**	`auctionEndNotifier.js` chạy định kỳ để notify về auction hết hạn nhưng **không dùng** `resolveAuctionStatus()` (hàm chưa tồn tại). Thay vào đó, nó tự tính lại điều kiện auction state theo cách riêng của mình — tạo ra một nguồn sự thật thứ ba về auction state (ngoài `GET /detail` và `GET /complete-order`).	Ba nơi tính auction state ngầm định có drift: notifier notify "EXPIRED" trong khi trả về "PENDING" cho cùng một produc user nhận email sai. Khi business rule th (ví dụ: grace period 5 phút sau end_at), tìm và sửa tất cả 3 nơi.

Tổng kết theo Nguyên tắc

Nguyên tắc	Số lần vi phạm	Mức độ nghiêm trọng
SRP	4 (STT 1, 3, 9, 13)	🔴 Cao
OCP	2 (STT 5, 6)	🟠 Vừa
LSP	*(không phát hiện vi phạm rõ ràng)*	—
ISP	*(được phản ánh một phần qua `review.model.js` dual API)*	🟠 Thấp
DIP	1 (STT 8)	🟠 Vừa
KISS	2 (STT 4, 13)	🔴 Cao
DRY	4 (STT 2, 7, 11–12, 14)	🔴 Cao
YAGNI	2 (STT 10, 11–12)	🟠 Thấp

Thứ tự Refactoring Đề Xuất

```
Bước 1 → Tách auction-state.js + cập nhật auctionEndNotifier    [STT 2, 6, 14]  ← Risk thấp nhất
Bước 2 → Clean up review.model.js                               [STT 11, 12]    ← Isolated change
Bước 3 → Xóa dead code GET /bid-history                         [STT 10]        ← Trivial
Bước 4 → Tách bid-engine.js + auction.service.js                [STT 4, 5, 8]   ← Core refactor
Bước 5 → Tách order.service.js + fix Order creation timing      [STT 9]         ← Domain boundary
Bước 6 → Tách notification.service.js + email template          [STT 7]         ← Reduce DRY
Bước 7 → Tách route files + move order/rating routes            [STT 1, 3, 13]  ← Final cleanup
```